PHY4605 Computational Methods in Physics
Learning Note | Week 12

Method Selection: Audit a computation and prepare a reproducible capstone

Recognise the learned method, audit supplied code, and assemble reproducible evidence that can be explained in an individual defence.

Physical question. When a familiar physics problem is presented with a supplied script or numerical output, how do we identify the appropriate learned method, find a clear defect, validate one result, and prepare evidence for a bounded capstone investigation?

Learning outcomes

1. match a familiar physical question to the learned method, required input, output, and one key limitation;
2. inspect and trace a supplied or AI-assisted MATLAB script, then choose and justify one validation check; and
3. assemble capstone evidence for the model, pseudocode, one modification, output, validation, interpretation, limitation, and individual defence.

How to use this note

Read the physical question before looking at the code or result. For each case, state the input, output, units, algorithm, and check in plain language. This week does not add a new required numerical method. It brings the methods from Weeks 01–11 together so that a result can be inspected and defended rather than merely displayed.

The course chain. *Physical model → scale and units → discretisation or data arrangement → algorithm → code → error or uncertainty → validation → physical interpretation.* A capstone packet should make this chain visible in a compact form.

Two small ideas help you read the evidence. **Discretisation** replaces a continuous input such as time by a finite list of points or steps; it makes the calculation runnable but introduces an approximation. **Error** is the difference between a numerical value and a known or reference value. **Uncertainty** is limited confidence caused by approximate inputs, sampling, or model assumptions. Report the unit and basis of any error or uncertainty rather than treating many displayed digits as proof.

1. Start from the output the question requests

Method selection is a physical match. First ask what the calculation must produce, then identify the learned method whose algorithm produces that output. A MATLAB function name is only a clue; the model, inputs, units, and limitations still need to agree.

Output sought	Familiar route	Input or evidence to inspect	One key limitation
Values of a known relation at many inputs	Array evaluation and plot	input array, output array, labels and units	the model and units must be appropriate
Several unknowns constrained together	Two-by-two linear system	coefficient matrix, source vector, reconstructed quantities	wrong equations can still give plausible numbers
The effect of changing one input	Parameter sweep	changed parameter array and held-fixed inputs	changing several inputs hides the cause
The value that makes a residual zero	Root finding	residual, bracket or supplied iteration	the root belongs to the stated equation and interval
A rate or gradient from nearby values	Numerical differentiation	sample spacing and derivative units	the estimate depends on the step size
An accumulated total or area	Numerical integration	samples, interval, and spacing	coarse sampling can change the result
A state as it changes in time	ODE simulation or Euler	initial state, rate law, timestep, and horizon	timestep and model assumptions limit confidence
Parameters inferred from measurements	Data fitting	data, fitted parameters, units, and residual pattern	a fit does not prove the model
An estimate from repeated random trials	Monte Carlo sampling	number of trials, output spread, and seed if supplied	random estimates vary with sample size
An output range from uncertain inputs	Sensitivity or uncertainty sweep	varied input, baseline, and held-fixed quantities	the range depends on stated assumptions

For example, a request for temperature at many times asks for a time trajectory, so a supplied ODE or Euler update is the natural route. If an exact expression is supplied, it is a reference for checking the numerical trajectory. The reference does not replace the method being assessed.

Recognition checkpoint. For any supplied case, say one sentence in this form: "The output is …, so I use …; its key input is …, and one limitation is …." Include the units of the output.

2. Audit supplied code before trusting its output

A script can run and still encode the wrong physics. Use the same short audit every time:

1. **Model:** write the physical law in words and check that the code has the same sign, terms, and assumptions.
2. **Units:** identify the unit of each input, intermediate rate, step, and output; check that additions combine like units.
3. **Arrays and indexing:** check the number of stored values, the first index, the current value, and the next value.
4. **Algorithm:** match each loop or calculation to one pseudocode step and verify that the update uses the intended current state.
5. **Output and validation:** confirm that the graph or table answers the question, then choose one check from a known value, trend, bound, reference, residual, refinement, or conservation relation.

Audit question	**Evidence in the script**	**What a defect can do**
Does the equation match the physical direction?	signs and terms in the rate or model line	produce a smooth but physically reversed trend
Do units match?	names such as `dt_s` and `rate_C_per_s`	make a numerical value look reasonable at the wrong scale
Does the index refer to the current state?	`state(n)` and `state(n+1)`	skip, overwrite, or repeat a state
Does the output answer the request?	selected endpoint, table, or labelled graph	report a convenient quantity instead of the required one

3. Trace the method before changing it

Tracing connects the algorithm to the numbers. Mark the first one or two iterations by hand before accepting an AI suggestion or editing a supplied line. For an update of the form

$$\text{next state} = \text{current state} + \text{step} \times \text{current rate},$$

identify the current state, calculate the rate with its units, multiply by the step, and then add the change. This small trace often exposes a wrong sign, a stale variable, a missing element-wise operator, or an incorrect loop bound.

Responsible AI record. If AI assistance is used, record the request, the part accepted, the part modified or rejected, and the independent check that supported the decision. A polished answer or complete chat history is not a validation record.

4. Worked example: Euler updates for Newton cooling

Physical picture and model

A hot object is placed in a cooler room. Its temperature falls quickly at first and then changes more slowly as it approaches the room temperature. The familiar Newton-cooling rate law is

$$\frac{dT}{dt} = -\frac{T - T_{\text{env}}}{\tau}.$$

The difference $T - T_{\text{env}}$ is a temperature difference. Its numerical value is the same in degrees Celsius and kelvin, so degrees Celsius may be used consistently here. The timescale τ must be in seconds because the rate is measured in degrees Celsius per second.

Quantity	Value	Meaning and unit
T_{env}	$20\,^{\circ}\text{C}$	constant environment temperature
$T_0 = T(0)$	$80\,^{\circ}\text{C}$	initial object temperature
τ	$100\,\text{s}$	cooling timescale
t_{final}	$200\,\text{s}$	end of the simulated horizon
h	$20\,\text{s}$ or $10\,\text{s}$	Euler timestep

Prediction checkpoint. The object starts at $80\,^{\circ}\text{C}$, cools toward $20\,^{\circ}\text{C}$, and should remain between those values for these timesteps. A result that rises away from the room temperature, crosses below it, or changes in the wrong direction needs an audit.

Euler's algorithm in words

Euler's method uses the current rate over one short interval:

$$T_{n+1} = T_n + h\left(-\frac{T_n - T_{\text{env}}}{\tau}\right).$$

The rate has units $^{\circ}\text{C}\,\text{s}^{-1}$. Multiplying it by h gives a temperature change in $^{\circ}\text{C}$, which can be added to T_n.

1. state T_{env}, T_0, τ, h, and the final time with units;
2. create time positions from 0 to t_{final};
3. reserve one temperature value for every time position and place T_0 in the first position;
4. for each current index, calculate the cooling rate from the current temperature;
5. store the next temperature using the Euler update;
6. repeat with $h = 20\,\text{s}$ and $h = 10\,\text{s}$, then compare the endpoints with the supplied exact reference; and
7. record one validation check, the physical interpretation, and one limitation.

Trace the first updates

For $h = 20\,\text{s}$, the initial rate is

$$\left.\frac{dT}{dt}\right|_{T=80} = -\frac{80 - 20}{100} = -0.6\,^{\circ}\text{C}\,\text{s}^{-1}.$$

The first two steps are therefore

$$T_1 = 80 + 20(-0.6) = 68\,^\circ\text{C}, \qquad T_2 = 68 + 20\left(-\frac{68-20}{100}\right) = 58.4\,^\circ\text{C}.$$

Here T_1 means the state after one Euler step. In MATLAB it is stored as `T_C(2)`, because `T_C(1)` stores the initial value T_0 at $t = 0$.

For $h = 10\,\text{s}$, the first two values are $T(10) = 74\,^\circ\text{C}$ and $T(20) = 68.6\,^\circ\text{C}$. These traces are quick evidence that the current temperature, not the original temperature, is used at every step.

Timestep	$T(0)$ (°C)	$T(20)$ (°C)	$T(40)$ (°C)	$T(200)$ (°C)
$h = 20\,\text{s}$	80.000	68.000	58.400	26.4425
$h = 10\,\text{s}$	80.000	68.600	59.366	27.2946

MATLAB starter code

The following starter code is self-contained. The names carry units, the first array entry stores the initial condition, and the loop writes the next state from the current state.

Listing 1: MATLAB starter code for Euler cooling: inputs, time grid, and initial state

```
T_env_C = 20;
T_initial_C = 80;
tau_s = 100;
t_final_s = 200;
dt_s = 20;                  % repeat with 10 s

nSteps = t_final_s / dt_s;
t_s = (0:nSteps) * dt_s;
T_C = zeros(size(t_s));
T_C(1) = T_initial_C;       % T(0)
```

The first temperature is at MATLAB index 1. At the first Euler update, T_1 is therefore written to index 2. The loop uses the current index n and writes the next index $n + 1$.

Listing 2: MATLAB starter code for Euler cooling: current-state rate and update

```
for n = 1:nSteps
    rate_C_per_s = -(T_C(n) - T_env_C) / tau_s;
    T_C(n+1) = T_C(n) + dt_s * rate_C_per_s;
end
```

Listing 3: MATLAB code for the cooling reference, endpoint error, and labelled output

```
T_exact_C = T_env_C + (T_initial_C - T_env_C) * exp(-t_s/tau_s);
endpoint_error_C = abs(T_C(end) - T_exact_C(end));

plot(t_s,T_C,'o-','LineWidth',1.2)
xlabel('Time, t (s)')
ylabel('Temperature, T (degrees Celsius)')
grid on
```

Run the three listings in order from the top after changing only `dt_s = 10`. The supplied exact reference for this model is

$$T_{\text{exact}}(t) = 20 + 60e^{-t/100},$$

so $T_{\text{exact}}(200\,\text{s}) = 28.1201169942\,^\circ\text{C}$. The exact expression is used as a reference check; students do not need to derive it for the required route.

h (s)	**Euler $T(200\,\text{s})$ (°C)**	**Exact reference (°C)**	**Absolute endpoint error (°C)**
20	26.4425	28.1201	1.6777
10	27.2946	28.1201	0.8255

A clear plus-sign defect

The rate law contains a minus sign for a reason. When $T > T_{\text{env}}$, the difference $T - T_{\text{env}}$ is positive, so the minus sign makes the rate negative and the object cools. If the object is colder than the environment, the difference is negative and the same minus sign makes the rate positive, so the object warms toward the environment.

Listing 4: MATLAB defect example: a sign reversal in the Euler cooling rate

```
for n = 1:nSteps
    rate_C_per_s = +(T_C(n) - T_env_C) / tau_s;  % physical defect
    T_C(n+1) = T_C(n) + dt_s * rate_C_per_s;
end
```

With $h = 20\,\text{s}$, the defective line gives $T(20) = 92\,^{\circ}\text{C}$ instead of $68\,^{\circ}\text{C}$. The code is syntactically valid, but the sign makes the temperature move away from the environment. Restore `rate_C_per_s = -(T_C(n) - T_env_C) / tau_s;` and rerun from a fresh session. An initial-value check alone would not catch this defect because both scripts begin at $80\,^{\circ}\text{C}$; the cooling-direction check and the exact endpoint comparison do.

5. One required validation and its interpretation

For this worked example, select the supplied exact solution as the main validation because it gives a known temperature at the same endpoint. The numerical evidence is:

> **Required validation result.** Reducing the timestep from 20 s to 10 s changes the endpoint from 26.4425 to 27.2946 °C and reduces the absolute error from 1.6777 to 0.8255 °C against 28.1201 °C. The initial value and cooling direction are also physically consistent. For the stated model and horizon, the 10 s result is better supported by the supplied evidence.

The result is not a claim that every smaller timestep is automatically reliable. It shows what happens for this model and these two choices. The Euler approximation remains below the exact cooling curve at 200 s because each explicit step uses the rate at the beginning of its interval. The physical model also assumes a constant environment temperature, a uniform object temperature, and one constant cooling timescale. Airflow, changing material properties, or internal heat generation would limit that model.

Evidence	Question answered	Interpretation
Initial value	Does the trajectory start at T_0?	the first stored state is the stated physical condition
Cooling direction and bound	Does the object approach T_{env}?	the sign and trend agree with the model for this case
Exact endpoint reference	Does refinement improve the requested endpoint?	$h = 10\,\text{s}$ is closer than $h = 20\,\text{s}$ for the supplied horizon

6. Capstone studio: assemble evidence that can be defended

The capstone is a bounded investigation using an approved starter Live Script and problem space. Complete the evidence in the order below. The code may be supplied, completed, or AI-assisted; the reasoning and checks must remain visible.

Evidence item	What to record	Quick quality question
Model and units	physical question, assumptions, variables, parameters, outputs, and units	could another student identify what each value means?
Pseudocode	ordered steps from input to output and check	does each step have one clear purpose?
Method and trace	supplied method, relevant code lines, and one traced update or calculation	can each group member explain one code section?
One modification	one changed physical parameter or assumption, with other choices stated	is the change controlled and predicted before running?
Main output	one labelled graph or table that answers the question	is the requested quantity visible with units?
Required and chosen check	the lecturer-required check plus one chosen check	does the evidence support the numerical claim?
Interpretation and limitation	trend, physical meaning, and one condition that limits the conclusion	does the explanation go beyond describing the shape?
Reproducibility and AI record	clean-session steps and accepted, modified, or rejected assistance	could the group rerun and explain the final result?

A clean-session reproduction check

Use this short checklist before marking the evidence final in the submission record:

1. close or restart MATLAB and open the approved starter script;
2. run the complete script from the first section, with no hidden Workspace variables;
3. confirm that the final parameter or assumption modification is visible in the script;
4. confirm that the main graph or table has quantities, units, and a readable label;
5. rerun both the required and chosen validations and record the numerical evidence; and
6. save the final script and evidence together so another person can reproduce the result.

Final evidence state. Mark the packet *final in the submission record* only after the complete fresh-session rerun reproduces the main output and both checks, every evidence item is present, and each group member can explain a relevant code section. If a rerun or check is missing, leave the packet as draft and record the pending action or defect; do not label it final merely because the code runs.

Decision record example. "We asked AI to explain the supplied Euler loop. We accepted its description of the index roles, modified its proposed sign after checking the physical cooling direction, and rejected an unnecessary replacement solver. Our independent check was the supplied exact endpoint and the initial-value/trend check." This is the level of decision evidence needed; a full transcript is not required.

Individual defence rehearsal

Each group member should be able to point to one relevant code section and answer these prompts without reading a prepared paragraph:

1. What physical quantity does this variable represent, and what is its unit?
2. Which line implements the chosen method, and what does it do to the current state or data?
3. What would you predict if the changed parameter were increased or decreased?
4. Which validation check did you choose, and why is it appropriate to this output?
5. What limitation should a reader keep in mind when interpreting the result?

For cooling, identify T_n, explain the minus sign, trace $80 \to 68 \to 58.4\,^{\circ}\mathrm{C}$ for $h = 20\,\mathrm{s}$, and justify the exact endpoint check.

Guided extension: compare supplied evidence

Sometimes two supplied results can answer a closely related question. Compare them using the physical output, the stated inputs, and the evidence available. Do not add a new implementation burden simply because another method is shown.

Supplied comparison	**Compare**	**State**
Euler with $h = 20\,\mathrm{s}$ versus $h = 10\,\mathrm{s}$	endpoint, trend, and reference error	whether refinement supports the chosen result for the stated horizon
Bisection versus Newton for a supplied residual	root value and supplied iteration evidence	which result is supported by the bracket or residual check
Forward versus central difference from supplied values	derivative estimate and units	how the step choice affects the estimate
`ode45` output versus a supplied Euler trajectory	trajectory and endpoint evidence	which output is being used as a reference and why

The comparison is evidence-based and bounded. A smooth line, a smaller iteration count, or a familiar function name is not enough by itself.

Optional extension

For distinction-level extension, transfer the audit to an unfamiliar but supplied case with limited support, design a second independent validation check, or implement one optional extension. These activities are optional and not required for a defensible ordinary capstone submission.

Three takeaways

1. Choose a learned method from the physical output requested, then name its required input, output, and one limitation.
2. Audit model, units, arrays, and algorithm before trusting supplied or AI-assisted code; trace one part and perform one justified validation check.
3. A capstone becomes reproducible and defensible when its model, pseudocode, controlled modification, output, validation, interpretation, limitation, and AI decisions are recorded together.

Preparation for the practical and capstone studio. Bring the approved starter Live Script and be ready to state the physical question, variables and units, selected method, one controlled modification, the validation check required by the activity, and one limitation. For the capstone, record both the required and chosen checks. Each group member should practise tracing one code section and explaining the associated physical result.